

Video 1.4: Docker Security Basics and Daemon Configuration

Duration: 8:00

Format: PPT (20%) / Terminal (35%) / Code Editor (30%) / Browser (15%)

Resolution: 1920x1080 @ 30fps

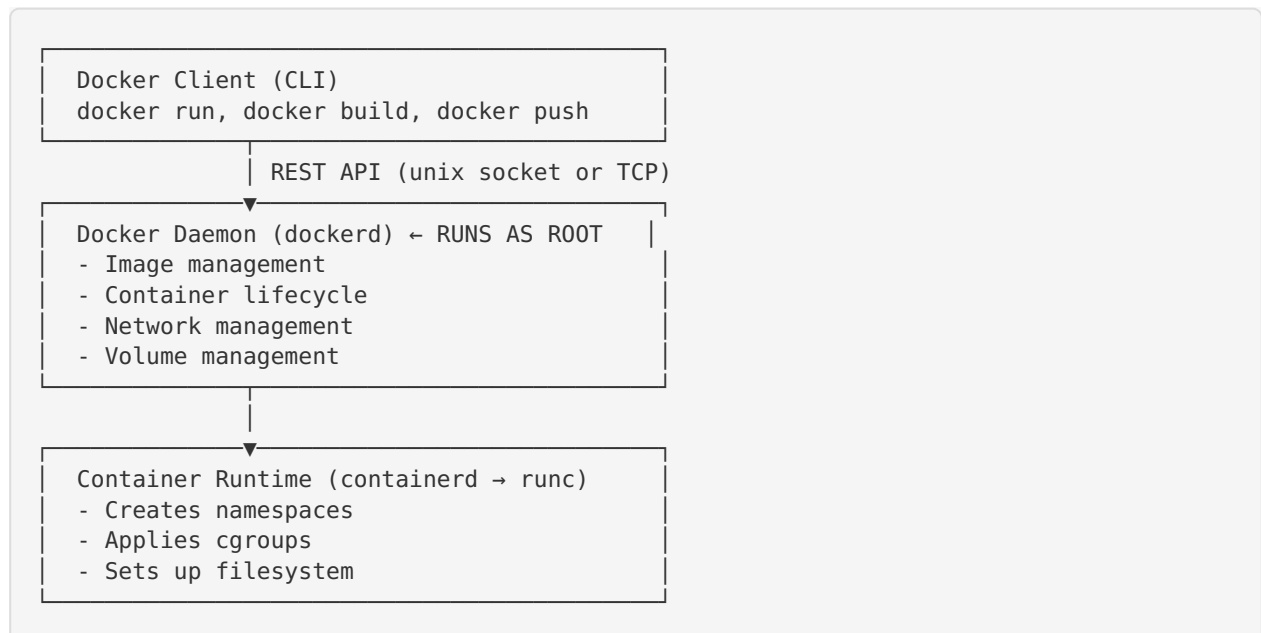
Prerequisites: Videos 1.1-1.3

SCRIPT BEGIN

[0:00-0:45] — Docker Architecture from a Security Lens (PPT Slide)

SLIDE: Title: "Docker Architecture — Security View"

Diagram showing:



NARRATION:

"Let's look at Docker from a security perspective. Docker has three main components. The CLI client — which sends commands. The Docker daemon — `dockerd` — which runs as root and manages everything: images, containers, networks, and volumes. And the container runtime — `containerd` and `runc` — which actually creates the isolated processes. The critical security fact is this: the Docker daemon runs as root. Anyone who can communicate with the daemon has effectively root-level access to the host. That communication happens over a Unix socket by default — and if you expose it over TCP, it's equivalent to giving root access to anyone who can reach that port."

[0:45-2:15] — The Docker Socket Risk (Terminal)

SCREEN: Terminal at 150% zoom

NARRATION:

“Let’s examine the Docker socket and understand why it’s the single most important security artifact on a Docker host.”

COMMAND + EXPLANATION:

```
# Check Docker socket permissions
$ ls -la /var/run/docker.sock
srw-rw---- 1 root docker 0 Jan 15 08:00 /var/run/docker.sock
```

“The Docker socket is owned by root, with group ownership by the `docker` group. Anyone in the `docker` group can send commands to the daemon. And since the daemon runs as root, being in the `docker` group is effectively equivalent to being root.”

```
# Prove it: mount the host filesystem from a container
$ docker run --rm -v /:/hostfs alpine cat /hostfs/etc/shadow
```

“Watch this. I’m in the `docker` group, so I can run this command. I mount the entire host root filesystem into a container, and then read `/etc/shadow` — the file containing password hashes. No `sudo` required. No privilege escalation. Just access to the Docker socket.”

```
# Even more dangerous: get a root shell on the host
$ docker run --rm -it -v /:/hostfs --privileged alpine chroot /hostfs bash
# whoami
root
# cat /etc/hostname
actual-host-name
```

“Now I’ve got a root shell on the host. I chrooted into the mounted host filesystem. I can modify any file, install backdoors, read credentials — anything. This is why Docker socket access IS root access, and why you should never add untrusted users to the `docker` group.”

[2:15-3:30] — Docker Daemon Configuration Audit (Code Editor)

SCREEN: Code editor at 150% zoom showing `/etc/docker/daemon.json`

NARRATION:

“Now let’s look at the daemon configuration file — this is where critical security settings live.”

CODE EDITOR — Insecure daemon.json (BAD):

```
{
  "hosts": ["unix:///var/run/docker.sock", "tcp://0.0.0.0:2375"],
  "icc": true,
  "iptables": true,
  "live-restore": true,
  "userland-proxy": true
}
```

NARRATION:

“This is a dangerously misconfigured daemon. Let me walk through the problems.”

“First — `tcp://0.0.0.0:2375` . This exposes the Docker API on all network interfaces over unauthenticated, unencrypted TCP. Anyone on the network can control your Docker daemon — and therefore your host. This is the number one Docker misconfiguration in the wild. Shodan consistently finds thousands of exposed Docker daemons on the internet.”

“Second — `icc: true` . Inter-container communication is enabled, meaning all containers can talk to each other over the Docker bridge network. In a security-conscious deployment, you want this disabled so containers are isolated by default.”

CODE EDITOR — Hardened daemon.json (GOOD):

```
{
  "hosts": ["unix:///var/run/docker.sock"],
  "tls": true,
  "tlscacert": "/etc/docker/ca.pem",
  "tlscert": "/etc/docker/server-cert.pem",
  "tlskey": "/etc/docker/server-key.pem",
  "tlsverify": true,
  "icc": false,
  "no-new-privileges": true,
  "userland-proxy": false,
  "live-restore": true,
  "log-driver": "json-file",
  "log-opts": {
    "max-size": "10m",
    "max-file": "3"
  },
  "default-ulimits": {
    "nofile": { "Name": "nofile", "Hard": 64000, "Soft": 64000 },
    "nproc": { "Name": "nproc", "Hard": 512, "Soft": 256 }
  },
  "storage-driver": "overlay2"
}
```

NARRATION:

“Here’s the hardened version. TCP exposure is removed — communication is Unix socket only. If you must expose TCP, TLS with mutual certificate verification is mandatory — that’s what the `tlsverify`, `tlscacert`, and `tlscert` settings do. ICC is disabled. `no-new-privileges` prevents processes from gaining additional privileges via `setuid` binaries or capability escalation. The userland proxy is disabled in favor of `iptables` — which is more efficient and provides better network isolation. Log limits prevent a container from filling the host disk. And `ulimits` restrict file descriptors and process counts.”

[3:30-4:30] — Scanning for Exposed Docker Daemons (Browser + Terminal)

BROWSER: Navigate to `https://www.shodan.io/search?query=port%3A2375+product%3A%22Docker%22`

NARRATION:

“Let me show you the real-world impact. On Shodan — a search engine for internet-connected devices — we can search for exposed Docker daemons on port 2375.”

“These are real, production systems with their Docker API exposed to the internet. An attacker can connect to any of these and immediately start containers, access volumes, and compromise the host. This is not hypothetical — it’s one of the most exploited misconfigurations in cloud environments.”

SWITCH TO TERMINAL:

```
# How an attacker checks for an exposed Docker daemon
$ curl -s http://target:2375/version 2>/dev/null | python3 -m json.tool
```

“If an attacker finds an exposed daemon, a simple curl request reveals the Docker version, API version, OS, architecture — and confirms they have full control.”

```
# Demonstrating remote container creation (EDUCATIONAL ONLY)
# This is what an attacker would do:
$ DOCKER_HOST=tcp://target:2375 docker run -d \
  -v /:/hostfs --privileged \
  alpine sh -c "cat /hostfs/etc/shadow > /tmp/exfil && sleep 3600"
```

“From anywhere on the internet, they can run a privileged container with the host filesystem mounted. Game over. This is why the very first Docker security check in any audit should be: is port 2375 or 2376 exposed to the network?”

[4:30-5:30] — Docker Security Defaults Audit (Terminal)**NARRATION:**

“Let’s audit what Docker does by default — and what it doesn’t.”

COMMAND + EXPLANATION:

```
# Check what capabilities Docker grants by default
$ docker run --rm alpine sh -c 'cat /proc/1/status | grep -i cap'
CapInh: 0000000000000000
CapPrm: 00000000a80425fb
CapEff: 00000000a80425fb
CapBnd: 00000000a80425fb
CapAmb: 0000000000000000
```

```
# Decode the capability bitmask
$ docker run --rm alpine sh -c 'apk add -q libcap && capsh --decode=00000000a80425fb'
```

“By default, Docker grants 14 Linux capabilities. These include CHOWN, DAC_OVERRIDE, FOWNER, KILL, NET_BIND_SERVICE, NET_RAW, SETFCAP, SETGID, SETUID, SYS_CHROOT, MKNOD, AUDIT_WRITE, and SETPCAP. Some of these are necessary for basic operations, but others — like NET_RAW, which allows raw socket creation — are routinely exploited for network attacks inside container environments.”

```
# Run a container with minimal capabilities
$ docker run --rm --cap-drop=ALL --cap-add=NET_BIND_SERVICE alpine id
uid=0(root) gid=0(root)
```

“Best practice: drop ALL capabilities and add back only what the application needs. Most web applications only need NET_BIND_SERVICE to listen on ports below 1024 — and if you run on port 8080 instead, you don’t even need that.”

[5:30-6:30] — Docker Content Trust (Terminal + Code Editor)

NARRATION:

“One more critical Docker security feature — image signing with Docker Content Trust.”

COMMAND + EXPLANATION:

```
# Check if Docker Content Trust is enabled
$ echo $DOCKER_CONTENT_TRUST
# (empty — disabled by default)
```

“Docker Content Trust is disabled by default. When disabled, Docker will pull any image from any registry without verifying its authenticity or integrity.”

```
# Enable Docker Content Trust
$ export DOCKER_CONTENT_TRUST=1
```

“When enabled, Docker will only pull images that are cryptographically signed. This prevents supply-chain attacks where an attacker pushes a malicious image to a registry.”

```
# Try pulling an unsigned image
$ docker pull someuser/unsigned-image:latest
# Error: remote trust data does not exist for ...
```

“With Content Trust enabled, unsigned images are rejected. This is a critical defense against supply-chain attacks — which have been used in real-world incidents to inject cryptominers and backdoors into production environments.”

CODE EDITOR — Docker Content Trust workflow:

```
# Content Trust Signing Workflow
# =====

# 1. Generate signing keys (first time)
$ docker trust key generate mykey

# 2. Add signer to repository
$ docker trust signer add --key mykey.pub myname registry.example.com/myapp

# 3. Sign and push an image
$ DOCKER_CONTENT_TRUST=1 docker push registry.example.com/myapp:v1.0

# 4. Verify signatures
$ docker trust inspect --pretty registry.example.com/myapp:v1.0

# 5. Enforce in daemon.json
# {
#   "content-trust": {
#     "mode": "enforced"
#   }
# }
```

“Here’s the complete workflow for setting up content trust. Generate keys, add signers, sign during push, and enforce in the daemon configuration. We’ll practice this hands-on in the lab exercises.”

[6:30-7:15] — Docker Bench Security (Terminal)

NARRATION:

“Finally, let me show you an automated tool for auditing Docker security.”

COMMAND + EXPLANATION:

```
# Run Docker Bench for Security
$ docker run --rm --net host --pid host \
  --userns host --cap-add audit_control \
  -e DOCKER_CONTENT_TRUST=$DOCKER_CONTENT_TRUST \
  -v /var/lib:/var/lib:ro \
  -v /var/run/docker.sock:/var/run/docker.sock:ro \
  -v /usr/lib/systemd:/usr/lib/systemd:ro \
  -v /etc:/etc:ro \
  docker/docker-bench-security
```

“Docker Bench for Security is the official CIS benchmark checker. It audits your Docker host, daemon configuration, container runtime, images, and Docker Swarm settings against the Center for Internet Security benchmarks. Run this regularly — and before any production deployment.”

```
# Sample output (abbreviated)
# [WARN] 2.1 - Run the Docker daemon as a non-root user
# [PASS] 2.2 - Ensure network traffic is restricted between containers
# [WARN] 2.3 - Ensure Docker is allowed to make changes to iptables
# [WARN] 4.1 - Ensure a user for the container has been created
# [PASS] 4.5 - Ensure Content trust for Docker is Enabled
```

“The output tells you exactly what passes and what needs attention. Every WARN is a security hardening opportunity.”

[7:15-8:00] — Key Takeaways and Bridge (PPT Slide)

SLIDE: Title: “Docker Security Essentials”

1. **Docker socket = root access** — protect it accordingly
2. **Never expose the daemon over TCP** without mutual TLS
3. **Drop all capabilities**, add back only what’s needed
4. **Enable Docker Content Trust** for supply-chain security
5. **Disable ICC** and use explicit network policies
6. **Audit with Docker Bench** against CIS benchmarks regularly

NARRATION:

“Remember these six principles. The Docker socket is root access — treat it like a root password. Never expose the daemon API without TLS and certificate authentication. Drop all capabilities by default. Enable Content Trust. Disable inter-container communication. And audit regularly with Docker Bench. In our final lesson for this module, we’ll step back and map the complete container attack surface — giving you the comprehensive threat model you need for the rest of this course.”

SLIDE: “Next: Container Attack Surface Mapping →”

SCRIPT END

Post-Production Notes

- Blur or redact any real results from Shodan to avoid showing actual vulnerable hosts
- The daemon.json comparison should use a split-screen or diff view with red (bad) and green (good) highlighting
- Docker Bench output should be shown scrolling, then paused on key WARN/PASS entries
- For the Docker socket demo, add an on-screen warning banner: “⚠ EDUCATIONAL PURPOSE ONLY”
- All commands should be pre-tested on Ubuntu 22.04 with Docker 24.x+
- The capability decode output should be shown with color-coded capabilities (risky = red, safe = green)